Stream: Internet Engineering Task Force (IETF)
RFC: 10055
Category: Informational
Published: October 2026
ISSN: 2070-1721
Author: M. Duke
Google

RFC 10055
Configuring UDP Sockets for Explicit Congestion Notification (ECN) for Common Platforms

Abstract

Explicit Congestion Notification (ECN) applies to all transport protocols in principle. However, it had limited deployment for UDP until QUIC became widely adopted. As a result, documentation of UDP socket APIs for ECN on various platforms is sparse. This document records the results of experimenting with these APIs in order to get ECN working on UDP for Chromium on Apple, Linux, and Windows platforms.

This document provides a snapshot of the ECN state of affairs as supported by these platforms at the time of writing. Readers should refer to the documentation of the various platforms for more up-to-date information.

Status of This Memo

This document is not an Internet Standards Track specification; it is published for informational purposes.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Not all documents approved by the IESG are candidates for any level of Internet Standard; see Section 2 of RFC 7841.

Information about the current status of this document, any errata, and how to provide feedback on it may be obtained at https://www.rfc-editor.org/info/rfc10055.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 4
2. Conventions and Definitions 5
3. Receiving ECN Codepoints 5
3.1. Setting the Socket to Report Incoming ECN Codepoints 5
3.1.1. Linux, Apple, and FreeBSD 5
3.1.2. Windows 5
3.2. Retrieving ECN Codepoints on Incoming Packets 6
3.2.1. Linux 6
3.2.2. Apple and FreeBSD 6
3.2.3. Windows 7
4. Sending ECN Codepoints 7
4.1. On a Per-Socket Basis 7
4.1.1. Apple, FreeBSD, and Linux 7
4.1.2. Windows 8
4.2. On a Per-Packet Basis 8
4.2.1. Apple, FreeBSD, and Linux 8
4.2.2. Windows 8
5. Security Considerations 8
6. IANA Considerations 9
7. References 9
7.1. Normative References 9
7.2. Informative References 9
Acknowledgments 10
Author's Address 10

1. Introduction

[RFC3168] defines a 2-bit field in the IP header for Explicit Congestion Notification (ECN), which provides network feedback to endpoint congestion controllers. Historically, this has mostly been relevant to TCP [RFC9293], where any incoming ECN codepoints are internally consumed by the kernel and, therefore, imply no application interface except for enabling and disabling the capability.

The Stream Control Transmission Protocol (SCTP) [RFC9260] has long supported ECN in its design. SCTP is sometimes carried over DTLS and UDP [RFC8261]. In principle, user-space implementers might have leveraged UDP ECN APIs to deliver ECN codepoints between SCTP and the UDP socket. At the time of publication, the IETF is not aware of any such efforts.

[RFC6679] defines ECN over RTP over UDP. The IETF is aware of a research implementation but cannot confirm any commercial deployments. [RFC8888] proposes a different feedback format to work with [RFC6679], and there are a few implementations of this.

QUIC [RFC9000] runs over UDP and has seen wider deployment than SCTP. The Low Latency, Low Loss, Scalable Throughput (L4S) architecture [RFC9330] and QUIC have combined to increase interest in ECN over UDP.

The Chromium Projects [CHROMIUM] provide a widely deployed protocol library that includes QUIC. An effort to provide ECN support for QUIC on the many platforms on which Chromium is deployed revealed that many ECN-related UDP socket interfaces are poorly documented.

This Informational document provides a record of that experience to encourage further support for ECN in other QUIC implementations and any consumer of ECN codepoints that operates over UDP. It is not a Standards Track document and does not bind platforms to any API nor suggest any such API.

Many socket APIs continue to reference the "ToS (Type of Service) byte", including the IP_TOS label, even though [RFC2474] obsoleted that in 1998. That 8-bit field now contains a 6-bit Differentiated Services Code Point (DSCP) and the 2-bit ECN field.

This document focuses on the APIs for the C and C++ languages. Other languages are likely to have different syntax and capabilities.

The "ecn-examples" code repository [EXAMPLES] is extremely compact code that can verify the information in this document.

This document addresses access to the ECN field in the IP header via socket APIs. It does not address UDP transport-layer options [RFC9868], which are a separate extension mechanism operating at a different layer.

2. Conventions and Definitions

This document is not a general tutorial on UDP socket programming and does assume familiarity with basic socket concepts like binding, socket options, and common system error codes.

Throughout this document, "Apple" refers to both macOS and iOS. "Linux" guidance also applies to Android.

3. Receiving ECN Codepoints

Network devices can change the ECN codepoint in the IP header. Since this feedback is required at the packet sender, the packet receiver needs to extract this codepoint from the UDP socket in order to report to the sender.

There are two components to this: 1) setting the socket to report incoming ECN marks and 2) retrieving the ECN codepoint for each incoming packet.

Note that Apple platforms additionally provide a framework for network connections that allows receiving ECN flags when using UDP without traditional socket option semantics. When sending or receiving UDP datagrams, IP protocol metadata carries ECN information in both directions. See [APPLE-NETWORK-FRAMEWORK].

3.1. Setting the Socket to Report Incoming ECN Codepoints

3.1.1. Linux, Apple, and FreeBSD

To receive ECN codepoints, applications set a socket option to "true" using a setsockopt() call.

On all platforms, IPv4 sockets require the IPPROTO_IP-level socket option with the name IP_RECVTOS to be set.

On all platforms, IPv6 sockets require the IPPROTO_IPV6-level socket option with the name IPV6_RECVTCLASS to be set.

If the IPv6 socket is not IPv6 only, on Linux hosts it is required to also set the IPPROTO_IP-level socket option IP_RECVTOS to receive ECN codepoints for UDP/IPv4 packets. This step is not necessary for Apple or FreeBSD.

At the time of writing, an example implementation can be found at [CHROMIUM-POSIX].

3.1.2. Windows

Windows documentation recommends using the function WSASetRecvIPEcn() to enable ECN codepoint reporting regardless of the IP version. This function dates to Windows 10 Build 20348, according to [WINDOWS-DOC].

However, this can also be accomplished by calling setsockopt() and using options of level IPPROTO_IP and name IP_RECVECN for IPv4 and level IPPROTO_IPV6 and name IPV6_RECVECN for IPv6. These options are documented at [WINDOWS-SOCKOPT].

For IPv6 sockets that are not IPv6 only, WSASetRecvIPEcn() will not enable ECN reporting for IPv4. This requires a separate setsockopt() call using the IP_RECVECN option.

If a socket is bound to an IPv4-mapped IPv6 address (i.e., it is of the format ::ffff:<IPv4 address>; see Section 2.5.5.2 of [RFC4291]), then calls to WSASetRecvIpEcn() return the error EINVAL. These sockets should instead use an explicit setsockopt() call to set IP_RECVECN.

At the time of writing, an example implementation can be found at [CHROMIUM-WINDOWS].

3.2. Retrieving ECN Codepoints on Incoming Packets

All platforms described in this document require the use of a recvmsg() call to read data from the socket and to retrieve the ECN codepoint, because that information is provided as ancillary data. Those platforms all return zero or more "cmsg"s that contain requested information about the arriving packet.

Examples of the technique described below can be found at [CHROMIUM-POSIX] and [CHROMIUM-WINDOWS].

3.2.1. Linux

If a UDP/IPv4 message is received, Linux will include a cmsg of level IPPROTO_IP and type IP_TOS. The cmsg data contains an unsigned char. This applies to IPv4 sockets and to IPv6 sockets that are not IPv6 only.

If a UDP/IPv6 message is received, Linux will include a cmsg of level IPPROTO_IPV6 and type IPV6_TCLASS. The cmsg data contains an int. This applies to IPv6 sockets.

The cmsg data contains the entire IP header byte, which includes the DSCP and the ECN codepoint. The ECN codepoint constitutes the two least significant bits of this byte.

The same applies to the Linux-specific recvmmsg() call.

3.2.2. Apple and FreeBSD

If a UDP/IPv4 message is received on an IPv4 socket, the ancillary data will contain a cmsg of level IPPROTO_IP and type IP_RECVTOS. The cmsg data contains an unsigned char.

If a UDP/IPv6 or UDP/IPv4 message is received on an IPv6 socket, the ancillary data will contain a cmsg of level IPPROTO_IPV6 and type IPV6_TCLASS. The cmsg data contains an int.

The cmsg data contains the entire IP header byte, which includes the DSCP and the ECN codepoint. The ECN codepoint constitutes the two least significant bits of this byte.

3.2.3. Windows

If the incoming packet is UDP/IPv4, the socket will include a cmsg of level IPPROTO_IP and type IP_ECN. The cmsg data contains an int.

If the incoming packet is UDP/IPv6, the socket will include a cmsg of level IPPROTO_IPV6 and type IPV6_ECN. The cmsg data contains an int.

The cmsg data solely consists of the ECN codepoint and requires no further bitwise operations.

4. Sending ECN Codepoints

Existing ECN specifications ([RFC3168] and [RFC9330]) envision a particular connection that is consistently sending the same ECN codepoint. It might transition that marking after successfully completing a handshake, recognizing that the path or the peer does not support ECN or transitioning to a new path. Therefore, using a socket option to configure a consistent marking is generally more resource efficient.

However, some server designs receive all incoming packets on a single socket. As the many connections that constitute this packet stream may have different support for ECN, it is suitable to provide the ECN codepoint on a per-packet basis.

Note that Apple platforms additionally provide a framework for network connections that allows sending ECN flags when using UDP without traditional socket option semantics. When sending or receiving UDP datagrams, IP protocol metadata carries ECN information in both directions. See [APPLE-NETWORK-FRAMEWORK].

4.1. On a Per-Socket Basis

4.1.1. Apple, FreeBSD, and Linux

For sending UDP/IPv4 packets on an IPv4 socket, Apple, FreeBSD, and Linux platforms allow the outgoing ECN codepoint to be configured by using the IPPROTO_IP-level socket option with the name IP_TOS. The value has the type int.

For sending UDP/IPv6 packets on an IPv6 socket, Apple, FreeBSD, and Linux platforms allow the outgoing ECN codepoint to be configured by using the IPPROTO_IPV6-level socket option with the name IPV6_TCLASS. The value has the type int.

For sending UDP/IPv4 packets on an IPv6 socket, Linux platforms allow the outgoing ECN codepoint to be configured by using the IPPROTO_IP-level socket option with the name IP_TOS.

For sending UDP/IPv4 packets on an IPv6 socket, Apple and FreeBSD platforms allow the outgoing ECN codepoint to be configured by using the IPPROTO_IPV6-level socket option with the name IPV6_TCLASS.

Except for Apple platforms, this setsockopt() call also sets the DSCP that comprises the rest of the header byte. Applications making this call ought to preserve any existing DSCP setting, which might require an additional getsockopt() call, to avoid overriding commands set by other code in the stack. If there are multiple threads making changes to this byte, further safeguards will be necessary.

An example of the technique described above can be found at [CHROMIUM-POSIX].

4.1.2. Windows

At the time of this writing, Windows does not provide a way to configure marking on a per-socket basis.

4.2. On a Per-Packet Basis

Packets can be individually marked with ECN codepoints using the ancillary data that accompanies a sendmsg() call.

4.2.1. Apple, FreeBSD, and Linux

For sending UDP/IPv4 packets on an IPv4 socket, Apple, FreeBSD, and Linux use a cmsg with level IPPROTO_IP and type IP_TOS. On Apple and Linux, the type of data is int; and for FreeBSD, it is unsigned char.

For sending UDP/IPv6 packets on an IPv6 socket, Apple, FreeBSD, and Linux use a cmsg with level IPPROTO_IPV6 and type IPV6_TCLASS. The type of the data is int.

For sending UDP/IPv4 packets on an IPv6 socket, Linux requires a cmsg with level IPPROTO_IP and type IP_TOS. Apple and FreeBSD accept a cmsg with level IPPROTO_IPV6 and type IPV6_TCLASS.

The same applies to the Linux-specific sendmmsg() call.

4.2.2. Windows

For IPv4 packets, Windows uses a cmsg with level IPPROTO_IP and type IP_ECN.

For IPv6 packets, Windows uses a cmsg with level IPPROTO_IPV6 and type IPV6_ECN.

An example of the technique described above can be found at [CHROMIUM-WINDOWS].

5. Security Considerations

The security implications of ECN are documented in [RFC3168] and [RFC9330]. This document is a guide to enabling these capabilities, which incurs no additional security considerations.

Note that implementing ECN capabilities on some platforms, but not others, can help peers identify the operating system in use by a host, which can have privacy implications. This document aims to mitigate that possibility.

6. IANA Considerations

This document has no IANA actions.

7. References

7.1. Normative References

[RFC2474] Nichols, K., Blake, S., Baker, F., and D. Black, "Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers", RFC 2474, DOI 10.17487/RFC2474, December 1998, <https://www.rfc-editor.org/info/rfc2474>.

[RFC3168] Ramakrishnan, K., Floyd, S., and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP", RFC 3168, DOI 10.17487/RFC3168, September 2001, <https://www.rfc-editor.org/info/rfc3168>.

[RFC9330] Briscoe, B., Ed., De Schepper, K., Bagnulo, M., and G. White, "Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture", RFC 9330, DOI 10.17487/RFC9330, January 2023, <https://www.rfc-editor.org/info/rfc9330>.

7.2. Informative References

[APPLE-NETWORK-FRAMEWORK] "NWProtocolIP.Metadata", <https://developer.apple.com/documentation/network/nwprotocolip/metadata>.

[CHROMIUM] "The Chromium Projects", <https://www.chromium.org/chromium-projects/>.

[CHROMIUM-POSIX] "udp_socket_posix.cc", commit 6bdc14d, 19 June 2026, <https://source.chromium.org/chromium/chromium/src/+/main:net/socket/udp_socket_posix.cc>.

[CHROMIUM-WINDOWS] "udp_socket_win.cc", commit 85a1d1c, 10 June 2026, <https://source.chromium.org/chromium/chromium/src/+/main:net/socket/udp_socket_win.cc>.

[EXAMPLES] "ecn-examples", commit 39e39bf, 1 December 2025, <https://github.com/nplab/ecn-examples>.

[RFC4291] Hinden, R. and S. Deering, "IP Version 6 Addressing Architecture", RFC 4291, DOI 10.17487/RFC4291, February 2006, <https://www.rfc-editor.org/info/rfc4291>.

[RFC6679] Westerlund, M., Johansson, I., Perkins, C., O'Hanlon, P., and K. Carlberg, "Explicit Congestion Notification (ECN) for RTP over UDP", RFC 6679, DOI 10.17487/RFC6679, August 2012, <https://www.rfc-editor.org/info/rfc6679>.

[RFC8261] Tuexen, M., Stewart, R., Jesup, R., and S. Loreto, "Datagram Transport Layer Security (DTLS) Encapsulation of SCTP Packets", RFC 8261, DOI 10.17487/RFC8261, November 2017, <https://www.rfc-editor.org/info/rfc8261>.

[RFC8888] Sarker, Z., Perkins, C., Singh, V., and M. Ramalho, "RTP Control Protocol (RTCP) Feedback for Congestion Control", RFC 8888, DOI 10.17487/RFC8888, January 2021, <https://www.rfc-editor.org/info/rfc8888>.

[RFC9000] Iyengar, J., Ed. and M. Thomson, Ed., "QUIC: A UDP-Based Multiplexed and Secure Transport", RFC 9000, DOI 10.17487/RFC9000, May 2021, <https://www.rfc-editor.org/info/rfc9000>.

[RFC9260] Stewart, R., Tüxen, M., and K. Nielsen, "Stream Control Transmission Protocol", RFC 9260, DOI 10.17487/RFC9260, June 2022, <https://www.rfc-editor.org/info/rfc9260>.

[RFC9293] Eddy, W., Ed., "Transmission Control Protocol (TCP)", STD 7, RFC 9293, DOI 10.17487/RFC9293, August 2022, <https://www.rfc-editor.org/info/rfc9293>.

[RFC9868] Touch, J. and C. Heard, Ed., "Transport Options for UDP", RFC 9868, DOI 10.17487/RFC9868, October 2025, <https://www.rfc-editor.org/info/rfc9868>.

[WINDOWS-DOC] "WSASetRecvIPEcn function (ws2tcpip.h)", 22 February 2024, <https://learn.microsoft.com/en-us/windows/win32/api/ws2tcpip/nf-ws2tcpip-wsasetrecvipecn>.

[WINDOWS-SOCKOPT] "MSDN - IPPROTO_IP socket options", 7 May 2026, <https://learn.microsoft.com/en-us/windows/win32/winsock/ipproto-ip-socket-options>.

Acknowledgments

The author would like to thank Ryan Hamilton, who provided constant advice through this effort. Randall Meyer from Apple and Nick Grifka from Microsoft provided useful hints about the behavior of their respective operating systems. However, the author takes full responsibility for any errors above.

Michael Tüxen wrote the "ecn-examples" code that was very helpful in verifying the conclusions in this document. He also made multiple editorial contributions.

Neal Cardwell, Gorry Fairhurst, Max Franke, Rodney Grimes, Will Hawkins, Guillaume Hétier, Max Inden, Jonathan Lennox, Colin Perkins, Marten Seemann, and Greg White made improvements to this document.

Author's Address

Martin Duke
Google
Email: martin.h.duke@gmail.com